GRANDE ECOLE

LA MANU

Création ■ Management ■ Numérique

Javascript

Les objectifs

Les objectifs

1. Tour d'horizon
2. Écrire des **instructions** en JavaScript
3. Déclarer et appeler des **fonctions** et pourquoi
4. Manipuler des **tableaux**
5. Le concept **d'objet**
6. Interagir avec le document **HTML** via le **DOM**
7. Manipuler les **dates**

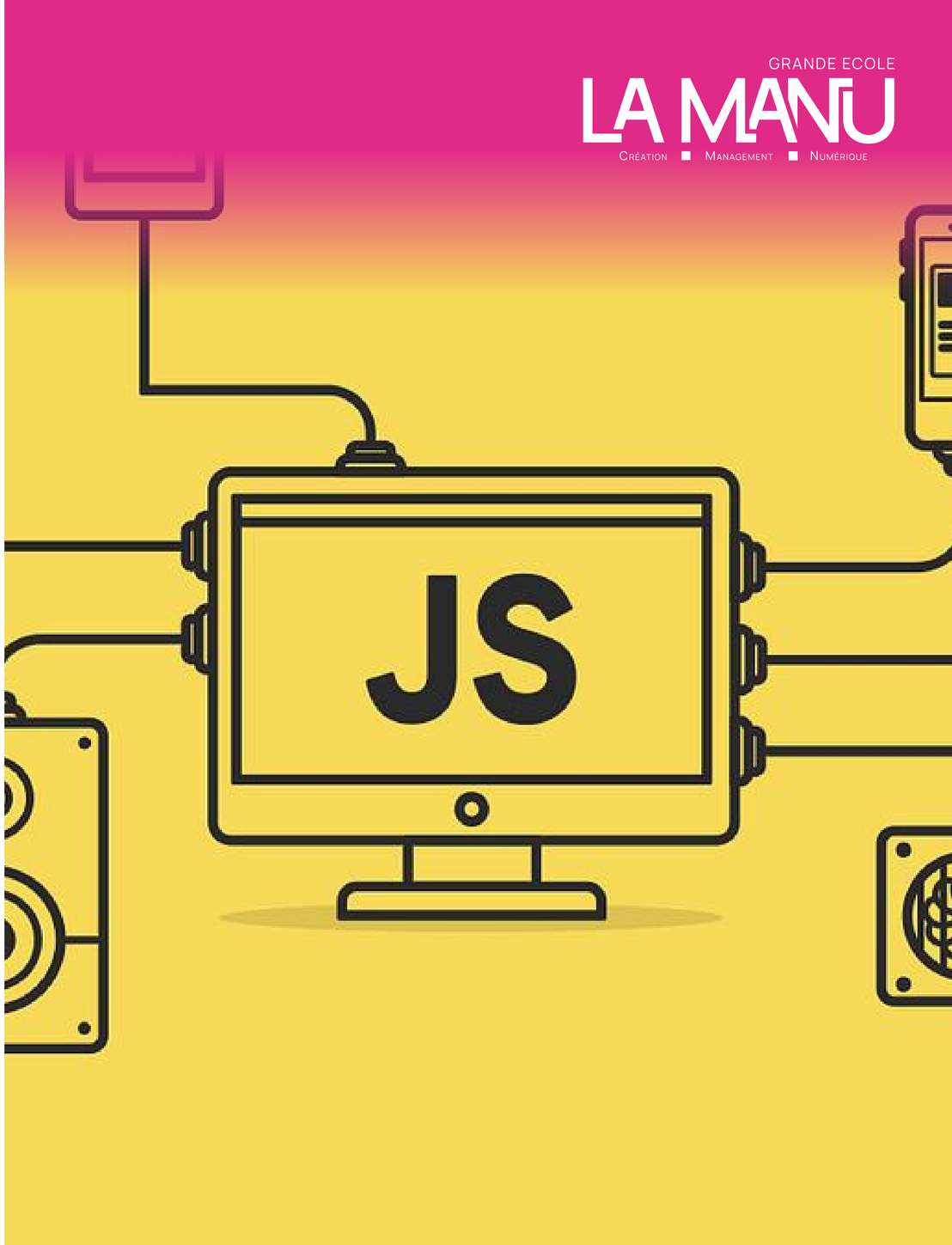
Introduction

L’histoire

Créer en 1995 par Brendan Eich, il ne lui a fallu que 10 jours, initialement appelé Mocha, puis Livescript.

Il a créer ce langage afin d’ajouter de l’interactivité aux pages web.

En 1997 les première spécification ECMAScript par l’**ECMA International**, cela permet à Javascript de devenir un standard.



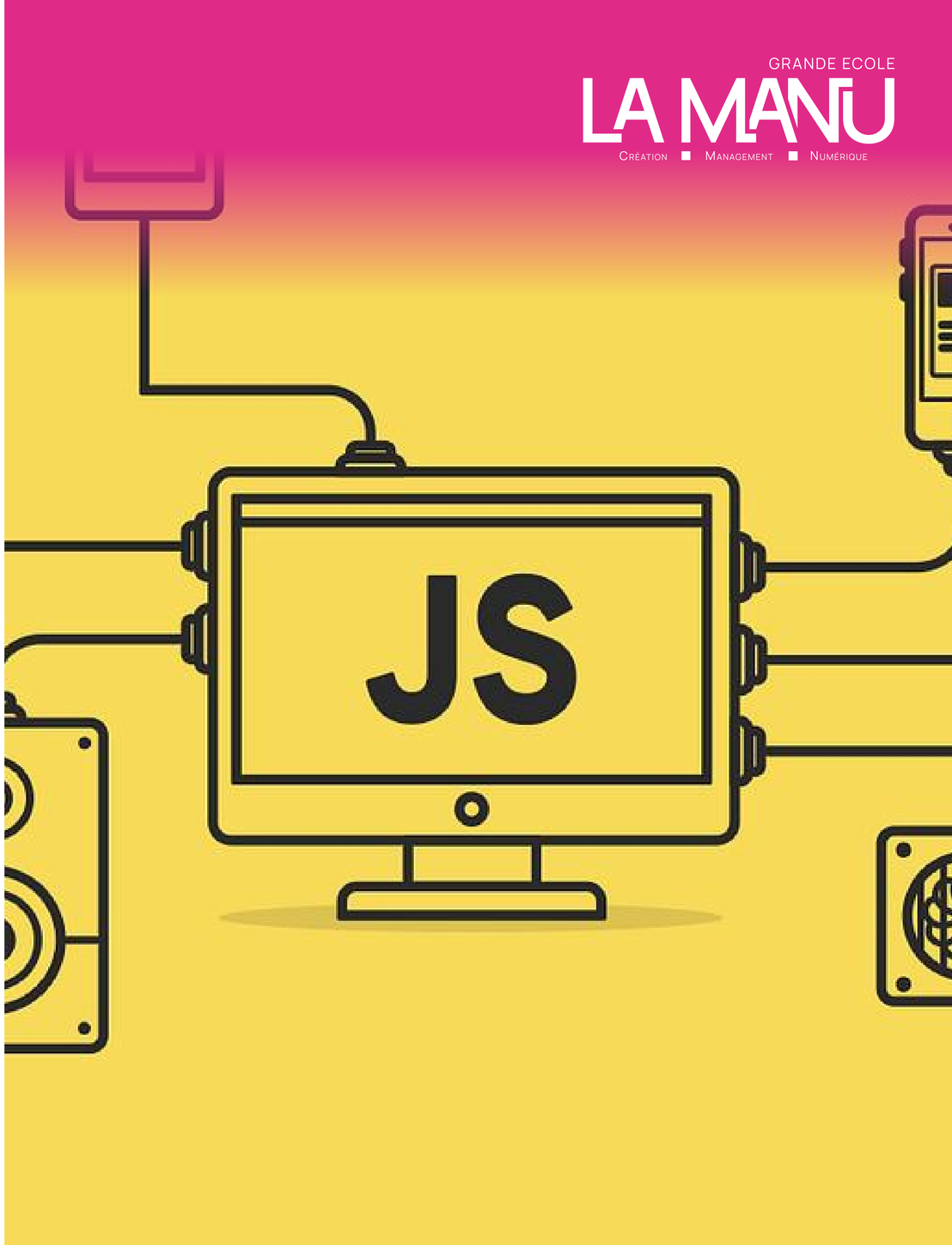
A partir des années 2000

Javascript devient de plus en plus populaire grâce à Ajax ou des bibliothèques comme JQuery, avec l'apparition des outils et des frameworks front-end.

En 2009, un certain Ryan Dahl a lancé **Node.js**, permettant ainsi d'utiliser Javascript pour le backend, sortant ainsi javascript du navigateur.

En 2015, une modernisation du Javascript a eu lieu avec l'arrivée de **ES6**.

Aujourd'hui, Javascript est utilisé pour le front-end, le back-end, les applications mobiles, les applications de bureau, les jeux, l'IoT et l'IA.



Les environnements

Navigateur (Client-side)

C’est l’environnement d’origine de Javascript.

Il permet dans cet environnement d’interagir avec la page HTML via le **DOM** (Document Object Model).

Par sécurité il est limité en accès, il ne peut pas lire directement des fichier ni accéder au système.

Fonctions clés :

- Modifier le contenu HTML/CSS dynamiquement
- Réagir aux évènements utilisateur
- Communiquer avec un serveur

Node.js (Server-side)

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

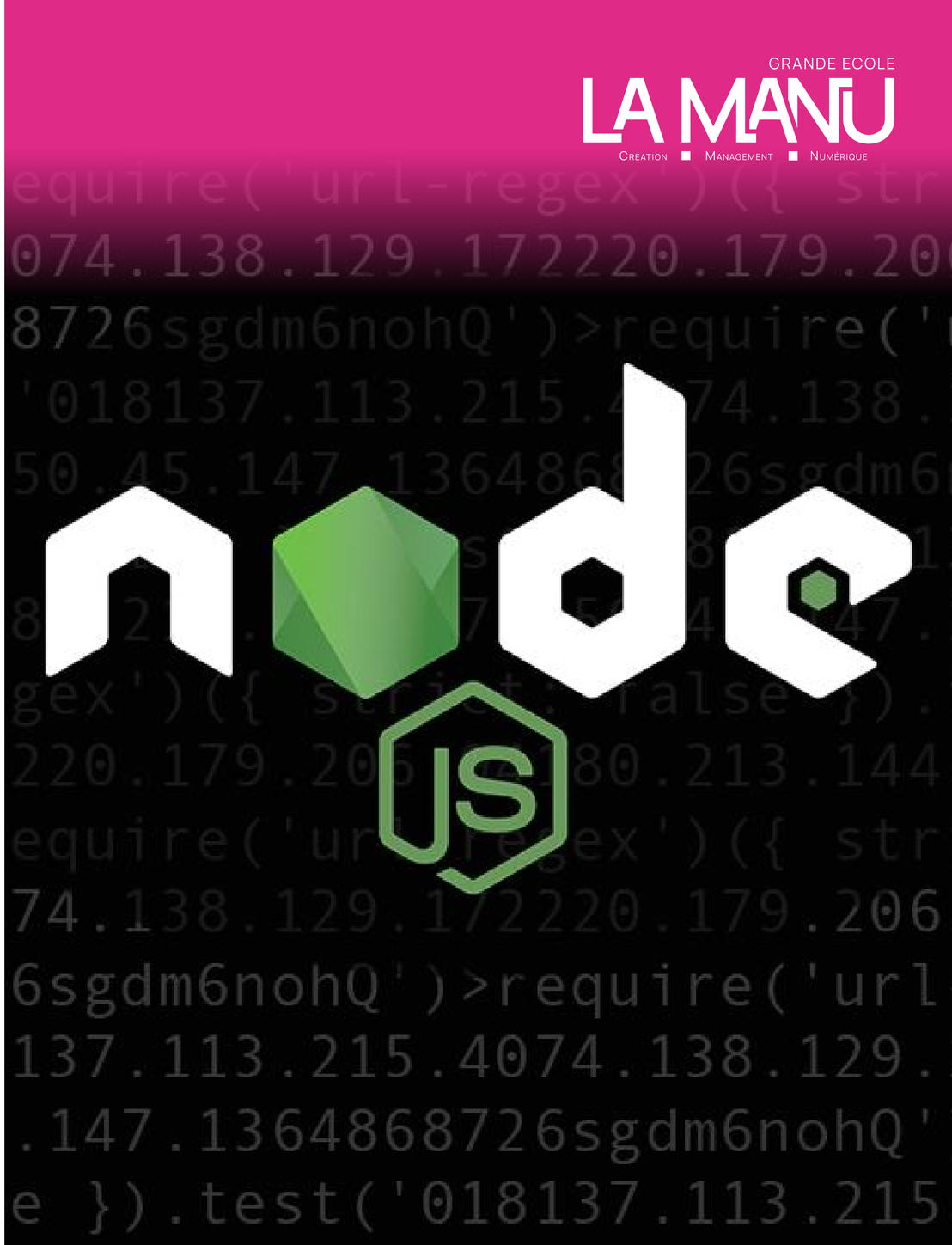
Un environnement hors navigateur.

Il utilise le moteur Javascript V8 (le même que celui de Chrome).

Il permet d'exécuter du Javascript sur un serveur ou en ligne de commande.

Fonctions clés :

- Lire/écrire dans les fichiers
- Gérer des serveurs HTTP
- Accéder au système
- Utiliser des modules
- Gérer des projets avec npm



Autres

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Javascript peut aussi être exécuté dans :

- Applications mobiles : React Native
- Applications desktop : Electron (ex. : Visual Studio Code)
- Objets connectés : Espruino, Tessel
- Navigateurs sans interface (headless) : Puppeteer, Playwright
- Navigateurs embarqués dans des outils de test : Cypress, Selenium

Les bases

Comment écrit on du JS ?

Les variables

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Une variable est un espace de stockage nommé en mémoire, utilisé pour garder une donnée que l'on souhaite réutiliser ou modifier.

let :

- Permet de déclarer une variable que l'on pourra modifier plus tard.

const :

- Permet de déclarer une constante : sa valeur ne peut pas être changer après l'affectation initiale.

var :

- Ancien mot-clé de déclaration (avant ES6).
- Il fonctionne encore, mais il est déconseillé aujourd'hui.

```
1   let prenom = "Alice";
2   const age = 20;
3   var notes = [15,20,18]
```

Initialisation et réaffectation

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Il est possible de déclarer une variable sans lui définir de valeur.

Si nous avons utilisé **let** nous pouvons lui attribué une valeur plus tard.
Il est aussi possible de changer sa valeur ensuite, autant de fois que nous le souhaitons.

```
1   let nom; //undefined
2   nom = "Claire";
3   nom = "Marie";
```

Bonnes pratiques de nommage

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Il existe un certains nombre de règles de bonnes pratiques afin d'avoir un code propre et exportable.
Parmis ces règles :

- Utiliser **l'anglais**
- Utiliser des nom **clairs et explicites**
- Ne pas commencer par un chiffre
- Pas de mot-clés réservés (let, if, function ...)
- Utiliser la convention **camelCase**

```
let function = "Chaine"; //Cela générera une erreur
let x = "Jean"; //Ce nom ne dit rien
let nombre = "Paul" //Ce n'est pas un nombre
let 1erPrix = 100;       // commence par un chiffre
let prénom-client = "";  // le tiret (-) n'est pas autorisé
let IsUserConnected = true; // PascalCase au lieu de camelCase
```

```
let hello = "Bonjour";
let firstname = "Jean";
let age = 10;
let firstPrice = 100;
let customerName = "Jane";
let isUserConnected = true;
```

Types de données

Que stocke-t-on dans les variables ?

Types primitifs

Les types primitifs sont les valeurs de base en Javascript.

Ces types sont dits immutables car leur contenu ne peut pas être modifié une fois créé.

Lorsqu'on assigne une valeur primitive à une variable, elle est stockée par valeur : Cela signifie que chaque variable reçoit une copie indépendante de la donnée.

Modifier l'une ne change pas l'autre.

Le type: string

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Représente une suite de caractères (texte), encadrée par " " ou ' '.

ES6 à introduit les templates literals (gabarits de chaînes).

Il s'écrive avec des backticks et permettent notamment :

- L'interpolation de variables
- L'écriture de chaines multi-lignes
- L'appel de fonctions de gabarit (template tags) pour un traitement avancé

```
1  let firstname = "Alice";
2  let message = 'Bienvenue';
3  let info = `Utilisateur ${firstname}`;
```

Le type: number

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

JavaScript n'a qu'un seul type numérique, qu'il soit entier (integer) ou à virgule (float).

Number gère des valeurs spéciales comme Infinity, -Infinity, NaN (Not a Number)

Il existe aussi des bonnes pratiques concernant Number:

- Il faut éviter les calculs sur des chaînes si le type n'est pas sûr.
- Utiliser Number() ou parseFloat() pour convertir un chaîne.

```
1  let age = 25;
2  let price = 19.99;
3  let total = -200;
4  console.log("abc" / 2); // NaN
5  console.log(1 / 0); //Infinity
```

Le type: boolean

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Le boolean permet de représenter deux états logiques : **true** ou **false**.

Utile pour faire des conditions, des boucles et d'autres choses...

En Javascript il existe des valeurs considérer comme automatiquement fausse (falsy values) :

- false | 0 | "" | null | undefined | NaN

Il existe une bonne pratique qui demande de nommer les variables booléennes avec un préfix clair : is, has, can, should etc..

```
1  let isLogged = true;
2  let havePoints = false;
3  let isValid = true;
4  let hasAccess = false;
```

Le type: undefined

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Une variable déclarée mais non initialisée contient undefined.

Particularité :

- typeof undefined renvoie "undefined"

Arrive souvent a cause d'un oublie ou d'une variable non encore définie.

```
1  let x;
2  console.log(x) //Undefined
```

Le type: null

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Représente l'absence intentionnelle de valeur ou un champ vide.

Particularité :

- typeof null renvoie "object" (bug historique)
- Utilisé pour dire "aucune valeur pour l'instant"

```
1  let result = null;
2
3  console.log(result) //null
```

Les opérateurs

Que peux t’on faire avec tout ca ?

Introduction

Les opérateurs sont des symboles ou des mots-clés qui permettent de manipuler des valeurs et de réaliser des opérations sur des variables.

Ils sont essentiels pour écrire des instructions logiques, effectuer des calculs, comparer des données, ou modifier le contenu des variables.

Javascript propose plusieurs catégories d'opérateurs, notamment :

- les opérateurs arithmétiques pour les calculs,
- les opérateurs d'assignation pour affecter des valeurs à des variables,
- les opérateurs de comparaison pour tester des conditions.

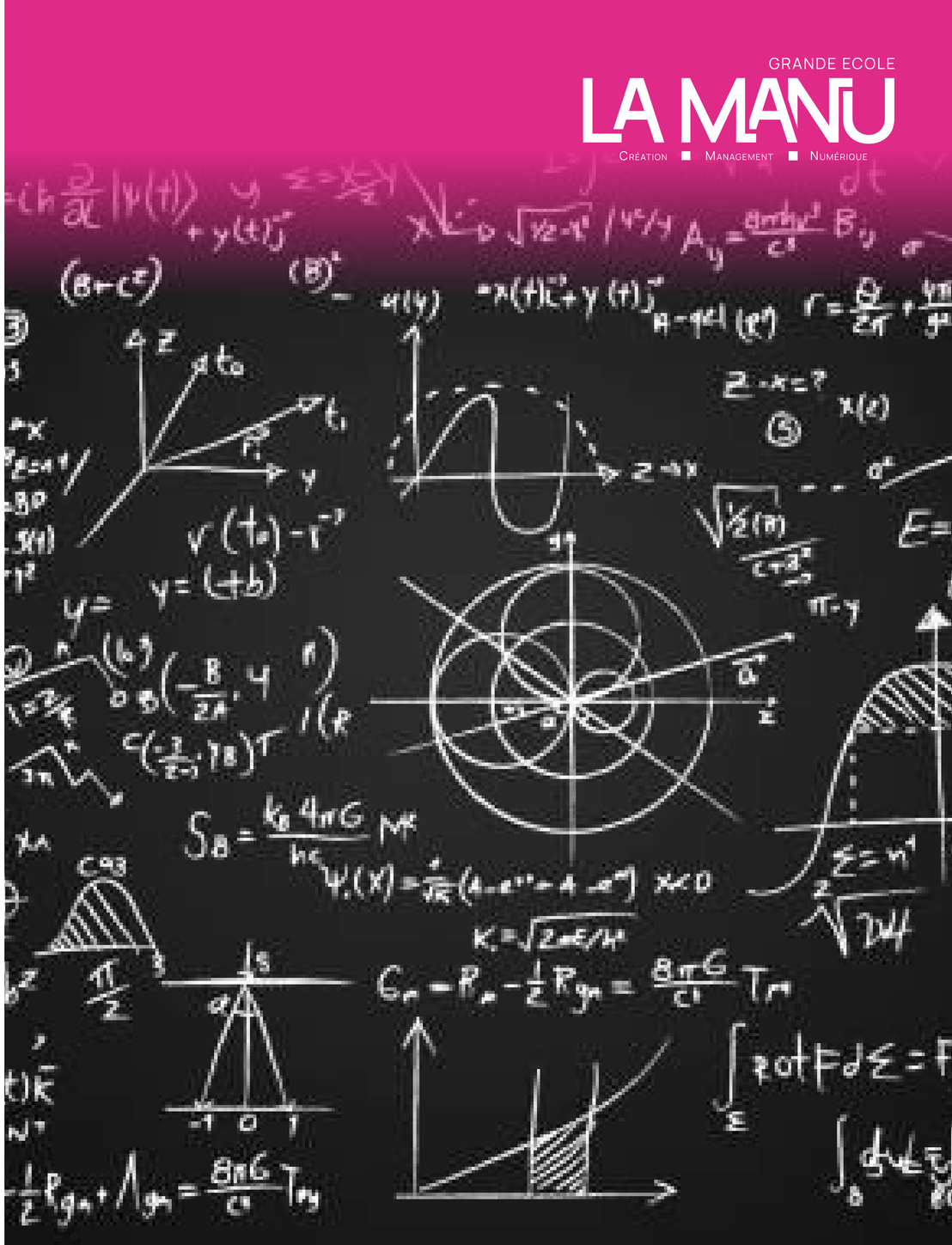
Opérateurs arithmétiques

Ces opérateurs permettent d'effectuer des opérations mathématiques classiques sur des nombres.

- **+** pour l'addition,
- **-** pour la soustraction,
- ***** pour la multiplication,
- **/** pour la division,
- **%** pour le modulo (le reste d'une division),
- ****** pour l'exponentiation (exemple : 2 ** 3 donne 8),
- **++** et **–** pour l'incrémentation et la décrémentation d'une variable.

ATTENTION : l'opérateur + peut aussi servir à concaténer des chaînes de caractères.

```
1   let name = "Jean";
2   let age = 10;
3   let sentence = name + " à " + age + "ans";
4   //Jean à 10 ans
```



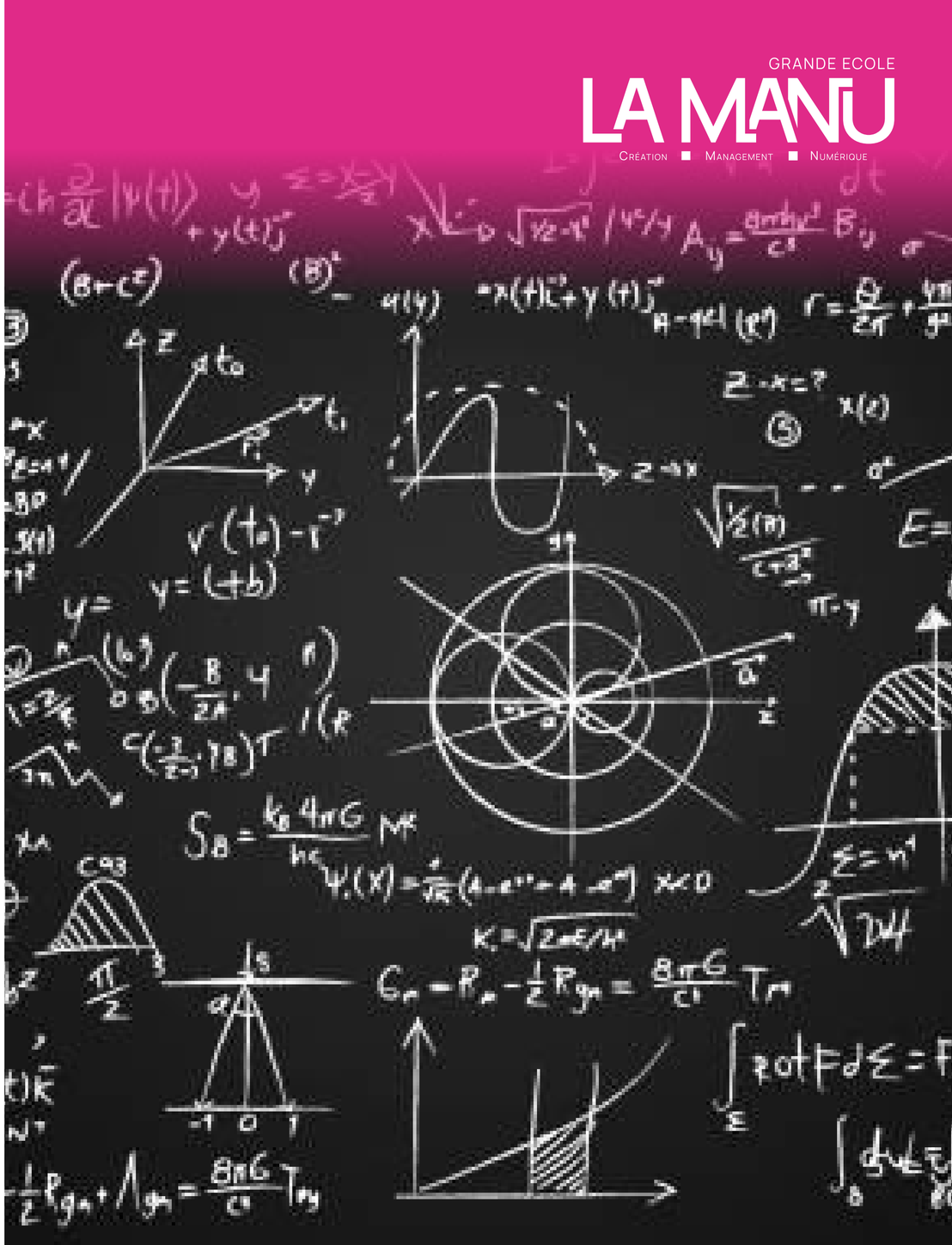
Opérateurs d'assignation

Ils permettent de donner une valeur à une variable, souvent en combinaison avec une opération.

Le plus simple est `=`, qui affecte une valeur. D'autres formes combinées permettent de raccourcir l'écriture, comme :

- `+=` (ajoute une valeur et assigne le résultat),
- `-=` (soustrait et assigne),
- `*=` (multiplie et assigne),
- `/=` (divise et assigne),
- `%=` (modulo et assigne).

```
1   //Les deux lignes donnent le même résultat
2   x += 3
3   x = x + 3
```



Opérateurs de comparaison

Ils sont utilisés pour comparer des valeurs, notamment dans les conditions.

Ils retournent toujours une valeur **booléenne** (true ou false).

- `==` (égalité abstraite) : compare les valeurs après conversion de type si nécessaire,
- `===` (égalité stricte) : compare les valeurs et les types,
- `!=` et `!==` pour tester l'inégalité,
- `<`, `>`, `<=`, `>=` pour les comparaisons numériques classiques.

```
5 === "5" //false
5 == "5" //true

5 < 4 //false
5 > 4 //true
```

Les portées de variables

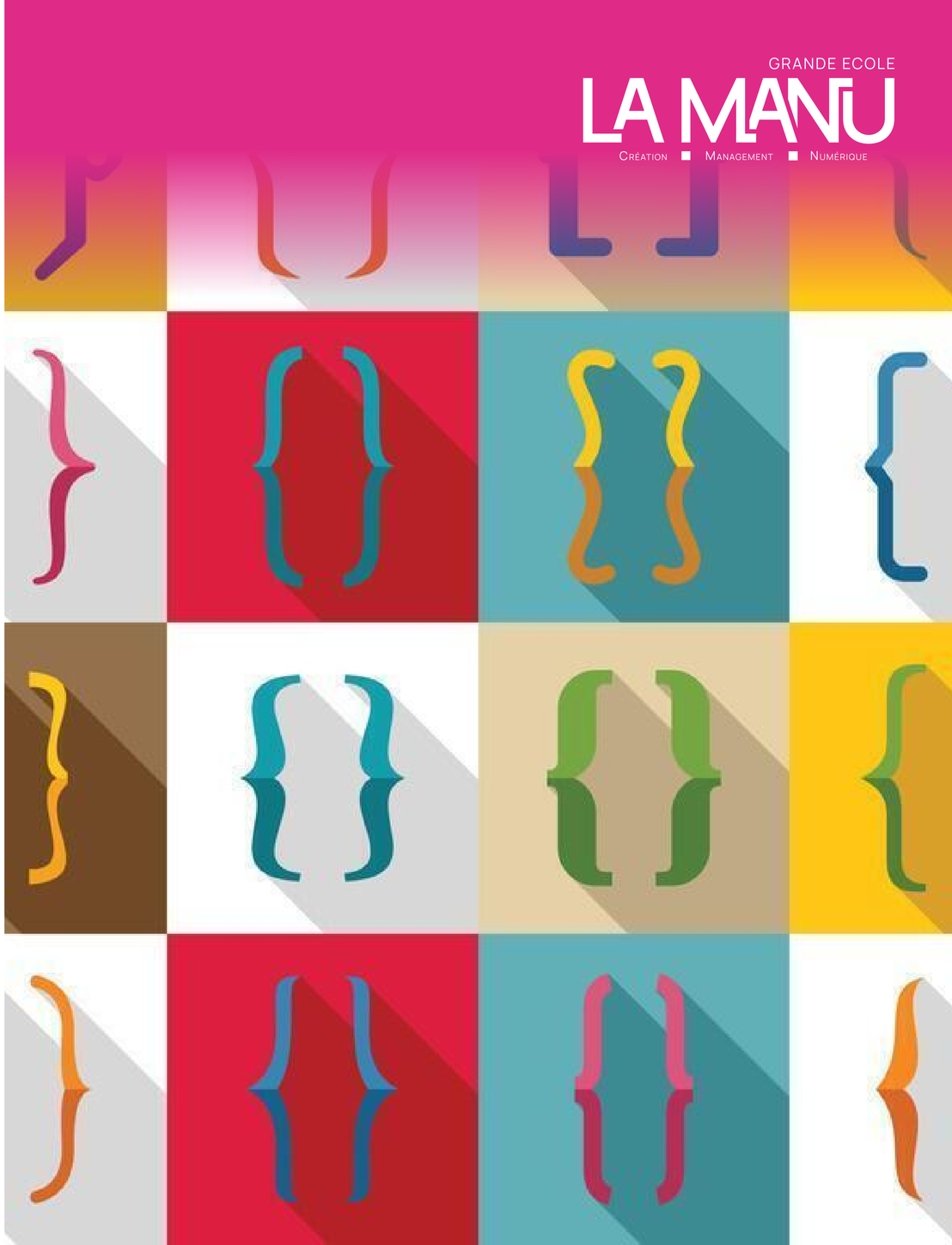
Les blocs

Avant de parlé de portée, nous devons déjà comprend ce que sont les blocs.

Un bloc est un ensemble d'instructions regroupées entre **accolades {}**.

On les utilise pour structurer le code et définir des **zones de portée**.
On les retrouve dans les conditions, les fonctions, les boucles etc ..

Chaque bloc crée son propre espace de variables.



Les portées ? (scope)

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

En Javascript, la portée d'une variable détermine où elle est accessible dans le code, dans quel blocs la variable sera disponible.

Depuis ES6, on distingue trois manières de déclarer une variable :

- var
- let
- const

Chacune a un comportement différent en termes de portée.

var : Portée fonction

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Déclarée avec var, une variable a une portée limitée à la fonction dans laquelle elle est définie.

Si elle est déclarée en dehors d'une fonction, elle devient **globale**.

Elle ignore les blocs et reste accessible en dehors du bloc.

```
1  {
2    {
3      var a = 42;
4    }
5    console.log(a); //42
6  }
```

Mais il faut éviter var ?

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Historiquement, **var** était le seul mot-clé pour déclarer des variables en Javascript.

Mais il présente plusieurs comportements **problématiques** :

- **Portée de fonction** seulement : contrairement à let et const, var ignore les blocs (if, for, etc.), ce qui peut provoquer des effets de bord inattendus.
- **Hoisting confus** : les variables var sont déplacées en haut de leur portée (hoisting), mais sans leur valeur. Cela peut conduire à des erreurs subtiles ou à des valeurs undefined.
- **Redéclaration possible** : une même variable peut être redéclarée plusieurs fois avec var, ce qui accroît les risques de conflits dans le code.

let, const: Portée de bloc

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Les mots-clés **let** et **const** ont une portée de bloc, ce qui signifie qu'ils ne sont accessibles que dans le bloc où ils sont définis.

Cette limitation évite les effets de bord et rend le code plus prévisible et permet de mieux contrôler notre script.

```
1  {
2      let a = 42;
3      const b = "John"
4      console.log(a, b); //Ok
5    }
6    console.log(a, b); //erreur
7  }
```

Structures de contrôle

Oui ou non ? - Les conditions

Conditions

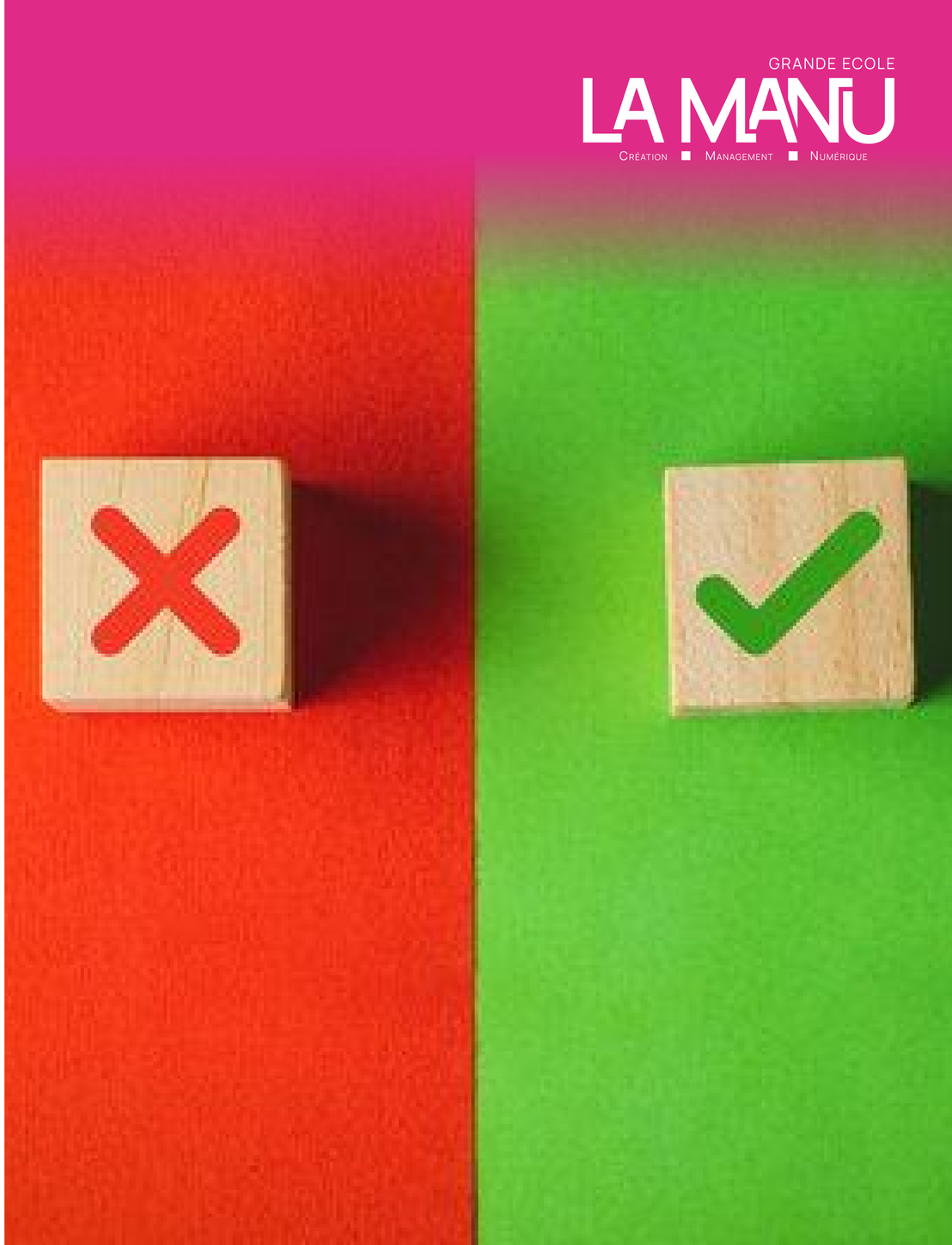
Les structures conditionnelles sont essentielles en programmation.

Elles permettent à un programme de prendre des décisions.

Concrètement, cela signifie qu'on peut exécuter des actions différentes selon que certaines conditions sont vraies ou fausses.

Javascript propose plusieurs façons d'écrire ces conditions, allant du classique **if/else** jusqu'au **switch** ou à des formes plus compactes comme l'opérateur **ternaire**.

Ces outils rendent votre code plus dynamique et réactif à différentes situations.



If/else

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Cette structure est la plus courante pour tester une ou plusieurs conditions successives.

- **if** : permet de tester une première condition. Si elle est vraie, le bloc de code associé est exécuté.
- **else if** : permet d'ajouter une ou plusieurs alternatives si la première condition est fausse.
- **else** : s'exécute par défaut, quand aucune des conditions précédentes n'est remplie.

Cette structure est idéale pour gérer des cas multiples avec des priorités ou des seuils, comme des évaluations, des niveaux d'accès, ou des tarifs selon l'âge, etc.

```
1  let note = 14;
2
3  if(note >= 16){
4    //afficher "Très bien"
5  } else if(note >= 10){
6    //afficher "Passable"
7  } else {
8    //afficher "Echec"
9  }
```

Switch

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

La structure switch est utile lorsqu'on veut comparer une même valeur à plusieurs cas possibles.

C'est une alternative plus lisible à une longue série de if...else if.

- **case** : chaque cas représente une valeur possible.
- **break** : empêche que le code continue à s'exécuter dans les cas suivants (comportement appelé fallthrough).
- **default** : optionnel, il définit ce qu'il faut faire si aucun cas ne correspond.

Le switch est particulièrement adapté quand on a de nombreux cas à tester sur une même variable, comme des menus, des codes d'erreur, ou des types d'objets.

```
1   let fruit = "pomme";
2
3   switch(fruit){
4     case "pomme":
5       //afficher "rouge ou verte"
6       break;
7     case "banane":
8        //afficher "jaune"
9        break;
10    default:
11      //afficher "fruit inconnu"
12  }
```

Quand utiliser switch ou if ?

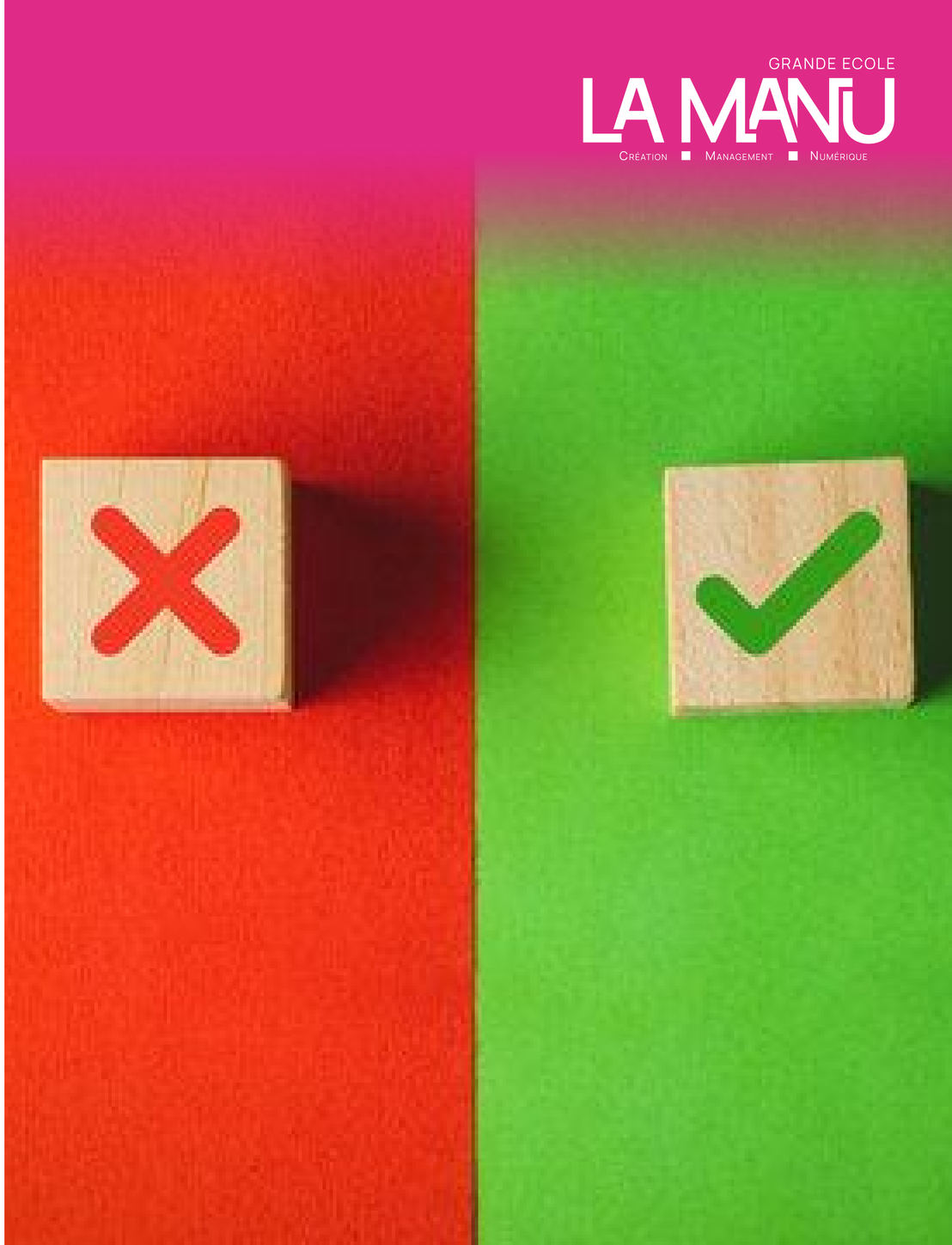
Les structures if et switch permettent toutes deux de gérer des conditions, mais elles ne sont pas toujours interchangeables.

Utilisez **if** quand :

- Vous testez des comparaisons complexes (ex : x > 10 && y < 20)
- Les conditions portent sur différentes variables ou expressions
- Vous avez peu de cas ou une logique plus souple

Utilisez **switch** quand :

- Vous comparez une seule variable à plusieurs valeurs **exactes**
- Vous voulez rendre votre code plus lisible qu'une suite de else if
- Les cas sont nombreux et bien définis (ex : types d'événements, jours, rôles)



Opérateurs logiques

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Les opérateurs logiques permettent de combiner des conditions pour prendre des décisions plus complexes.
Ils renvoient toujours une valeur booléenne (true ou false).

- **&&** (ET logique) : la condition est vraie si les deux sont vraies.
  Ex : "majeur et en France"
- **||** (OU logique) : la condition est vraie si au moins une est vraie.
  Ex : "étudiant ou demandeur d'emploi"
- **!** (NON logique) : inverse une condition.
  Ex : !connecté signifie "non connecté"

Ces opérateurs rendent les tests plus puissants et flexibles.

```
1  let age = 20;
2  let inscrit = true;
3  if(age >= 18 && inscrit){
4    //Accès autorisé
5  }
6
7  let admin = false;
8  let moderateur = true;
9  if(admin || moderateur){
10   //Accès au dashboard
11 }
12
13 let connected = false;
14 if(!connected){
15   //Veuillez vous connecter
16 }
```

Combinaisons d’opérateurs

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Dans la vie réelle, les décisions sont rarement binaires.

Par exemple : “Je vais au cinéma si j’ai fini mes devoirs et s’il y a encore des places ou si j’ai une carte illimitée.”

Les combinaisons d’opérateurs logiques permettent de reproduire ce genre de logique complexe dans le code en combinant &&, || et !

```
1   let age = 17;
2   let parentPresent = true;
3   let ticket = true;
4   if((age >= 18 || parentPresent) && ticket){
5     //entrée autorisée
6    } else {
7     //entrée refusée
8    }
9
10   let isConnected = false;
11   let isAdmin = true;
12   if(!isConnected && isAdmin){
13     //Admin non connecté
14   }
```

Les ternaires

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

L'opérateur ternaire est une forme abrégée d'un if / else.
Il permet d'écrire des conditions simples en une seule ligne, ce qui rend le code plus concis et lisible.

Pourquoi utiliser le **ternaire** ?

- Pour simplifier les conditions courtes
- Pour écrire moins de lignes de code
- Pour affecter une valeur selon une condition

condition **?** valeurSiVrai **:** valeurSiFaux;

```
let age = 20;
let message = age >= 18 ? "Majeur" : "Mineur"
// Ici, la variable message prend une valeur différente selon la condition.
```

Ne l'utilisez que pour des cas simples !
Des ternaires imbriqués ou trop longs peuvent nuire à la lisibilité

```
//Moins lisible
let note = 17;
let mention = note > 16 ? "Très bien" : note >=12 ? "Bien" : "Passable"
```

Structures de contrôle

Encore ... et encore ... - Les itérations

Structure itérative

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Quand on programme, on est souvent amené à réaliser plusieurs fois une même opération.

Par exemple :

- Afficher tous les éléments d'un tableau.
- Calculer la somme de plusieurs nombres.
- Répéter une action jusqu'à obtenir un résultat donné.
- Parcourir une liste d'utilisateurs pour appliquer une opération à chacun.

Au lieu d'écrire plusieurs fois le même code, on utilise les **structures itératives**, qui permettent d'automatiser et de répéter facilement ces opérations.

Comment on fait en JS ?

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

En Javascript il existe 3 boucles principales.

- La **boucle for**: Utilisée quand on connaît le nombre exact d'itérations à réaliser
- La **boucle while**: Utile quand on ne connaît pas forcément à l'avance le nombre d'itérations
- La **boucle do...while**: Comme while, mais le bloc s'exécute au moins une fois, car la condition est testée après l'exécution

La boucle for

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

La boucle **for** est la structure itérative la plus utilisée en Javascript.

Elle permet de répéter un bloc d'instructions un nombre précis de fois.

On la choisit surtout quand on connaît exactement combien de fois on veut faire tourner la boucle.

```
for(initialisation; condition; incrémentation){
  //code à répéter
}
```

```
for(let i = 0; i < 5; i++){
  //Tour numéro i
}
/*
- i commence à 0
- Tant que i est strictement inférieur à 5, la boucle continue
- Après chaque itération, on ajoute 1 à i (i++)
- Résultat : affichage des tours de 0 à 4 (5 tours)
*/
```

Conseils & bonnes pratiques

Le compteur i est souvent utilisé pour parcourir les éléments d'un tableau grâce à leurs index.

On peut modifier le pas d'incrémentation, par exemple i += 2 pour sauter des valeurs.

Il ne faut pas oublier l'incrémentation, sinon la boucle peut devenir infinie.

On peut déclarer plusieurs variables dans l'initialisation, par exemple :

```
1   for(let i, j=10; i < 5; i++, j--){...}
```

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

La boucle while

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

La boucle **while** exécute un bloc tant qu'une condition est vraie.

Elle est particulièrement adaptée lorsque l'on ne connaît pas à l'avance le nombre d'itérations.

```
while (condition) {
  // code à répéter
}
```

```
let i = 0;
while(i < 5){
  //Tour numéro i
  i++;
}
/*
- Avant chaque exécution, la condition i < 5 est testée.
- Si elle est vraie, on exécute le bloc, puis on incrémente i.
- Si elle est fausse, on sort de la boucle.
*/
```

Conseils & bonnes pratiques

Initialiser les variables avant la boucle.

Modifier les variables à l'intérieur de la boucle pour que la condition devienne fausse un jour (éviter les boucles infinies).

Pratique pour lire des données jusqu'à ce qu'une condition soit remplie (exemple : attendre une saisie utilisateur valide).

La boucle do...while

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Le do...while est une variante du while où le bloc de code est exécuté au moins une fois, avant que la condition ne soit testée

```
do {
  // code à répéter
} while(condition);
```

```
let i = 0;
do {
  //Tour numéro i
  i++;
} while(i < 5);
/*
- Ici, la boucle s'exécute une première fois sans tester la condition.
- Puis la condition est testée, et la boucle continue ou s'arrête selon le résultat.
*/
```

Conseils & bonnes pratiques

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Utiliser do...while lorsque l'on veut forcer au moins une exécution.

Utile dans les cas où on demande une saisie utilisateur, et qu'on veut répéter tant que la saisie n'est pas correcte.

```
1  //Exemple
2  let password;
3  do {
4    password = prompt("Entrez votre mot de passe (au moins 8 caractères)");
5  } while(password.length < 8);
```

Les fonctions

Je vais ranger ca là !

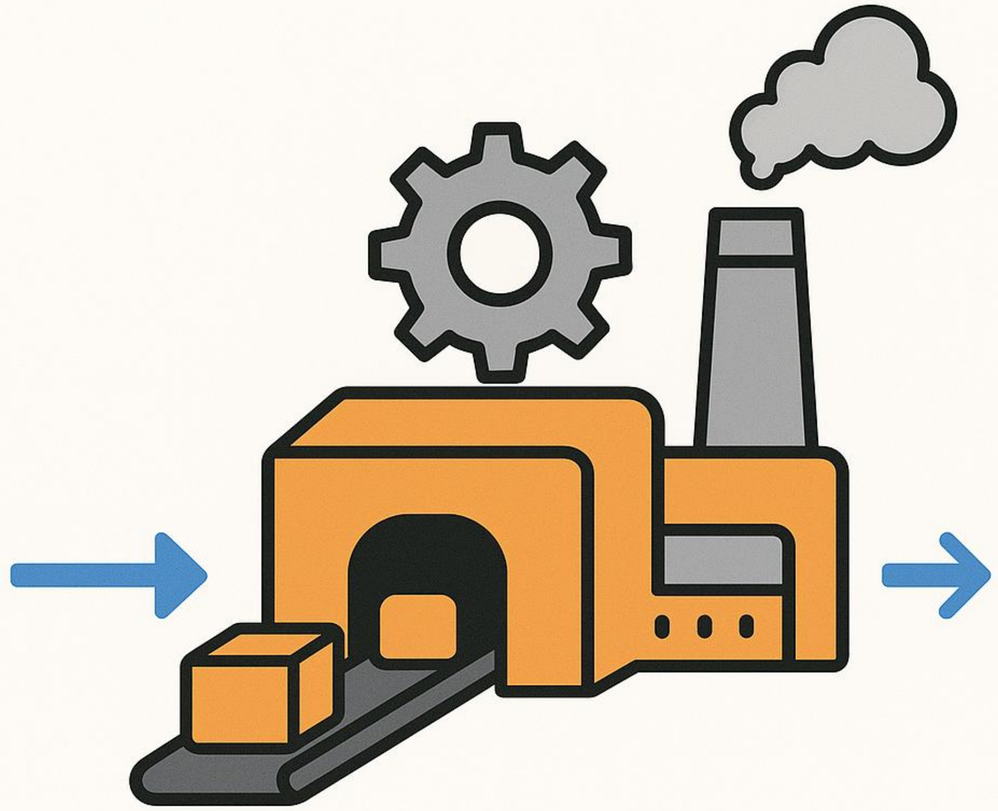
Pourquoi ?

Une fonction est un bloc d'instructions nommé, qu'on peut exécuter plusieurs fois dans un programme.

Cela permet d'écrire du code plus organisé, lisible et réutilisable.

Avantages principaux :

- Lisibilité
- Maintenance
- Modularité
- Réutilisation
- DRY (Don't Repeat Yourself)

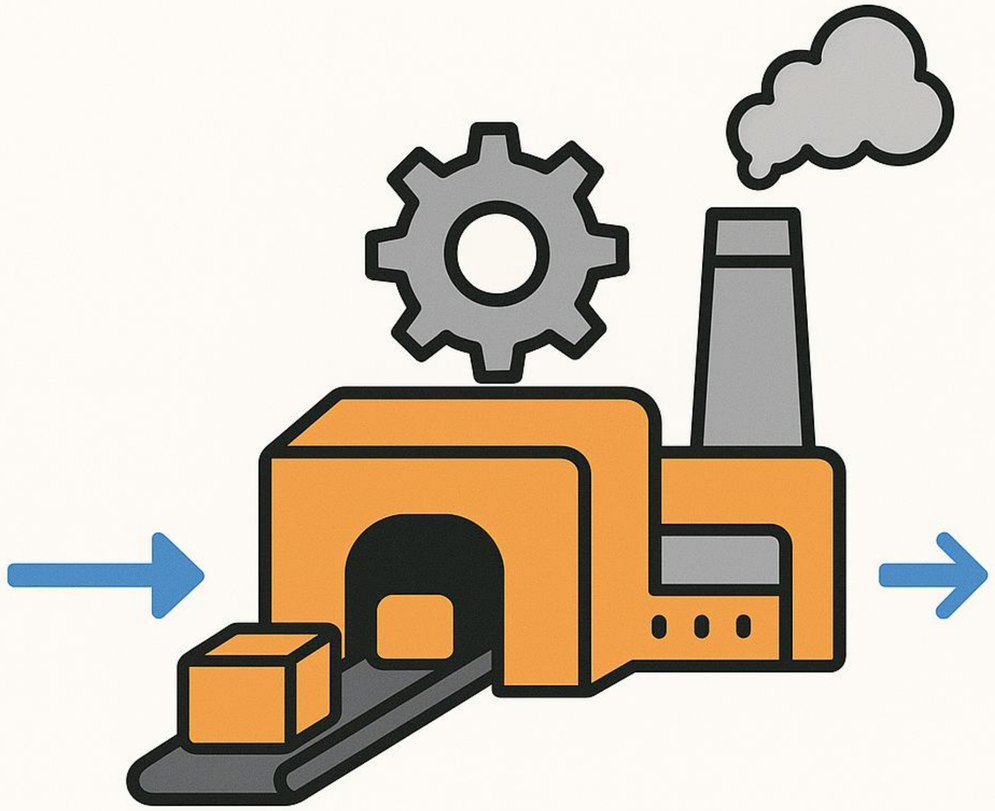


Mais comment ?

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Pour déclarer une fonction en Javascript il y a plusieurs méthodes possibles:

- Déclaration de fonction
- Expression de fonction
- Fonction fléchée



Fonctions classique

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Fonction nommée: meilleure lisibilité, debuggage plus clair.

Hoistée : Elle est "remontée" en mémoire pendant le parsing du code Javascript. On peut donc l'appeler avant sa définition dans le code source.

Les fonctions permettent de structurer le code plus librement.

Elles facilite l'écriture de fichiers modulaires ou organisés en ordre logique (ex. : regrouper les fonctions en bas du fichier)

```
1  sayHello(); // OK malgré
   qu'elle soit définie plus bas
2  function sayHello() {
3    //Afficher Bonjour
4  }
```

Précautions

- Si deux fonctions ont le même nom, la dernière écrasera les précédentes (comme toute redéclaration).
- Hoisting: Attention à ne pas en abuser dans des scripts très longs : il peut être plus clair de déclarer les fonctions proches de leur premier appel.

Expression de fonction

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

L'expression peut contenir une fonction sans nom (anonyme) ou parfois nommée pour le debug.

On assigne la fonction à une variable (ici sayGoodNight), cette variable sert à appeler la fonction.

Non hoistée: Contrairement à la déclaration classique, elle n'est pas levée lors du parsing, elle ne peut pas être utilisée avant sa définition.

La fonction n'est créée qu'au moment où la ligne est exécutée (et non pendant le parsing).

```
1  const sayGoodNight = function(){
2    //afficher Bonsoir
3  }
```

Fonctions fléchées

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Syntaxe courte et moderne: Introduites avec ES6, les fonctions fléchées offrent une écriture plus concise.

Pas de mot-clé function: On utilise => pour définir la fonction.

Pas de “this” propre: Le “this” est hérité du contexte d’exécution parent.

Très utilisée dans les callbacks: Notamment pour les fonctions courtes, promesses, tableaux, etc.

Utilisation typique dans les callbacks des méthodes :

- .map(),
- .forEach(),
- promesses...

```
1  const sayHello = () => {
2    //afficher "Salut"
3  }
```

Ok mais les parenthèses ?

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Quand on écrit une fonction, on veut souvent qu'elle puisse agir de manière différente selon les données qu'on lui donne.

C'est là que les **paramètres** entrent en jeu.

Les paramètres permettent de rendre une fonction dynamique.

Au lieu d'écrire plusieurs fois un même bloc de code pour des cas légèrement différents, on peut factoriser ce comportement dans une seule fonction.

Les termes utilisés:

- **Paramètre** : c'est un nom qu'on donne dans la définition de la fonction, comme une variable locale.
- **Argument** : c'est la valeur réelle qu'on envoie lors de l'appel de la fonction.

Paramètres et arguments

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Javascript permet beaucoup de flexibilité avec les paramètres :

- On peut appeler une fonction avec moins d'arguments que prévu → les paramètres restants vaudront undefined.
- On peut appeler une fonction avec plus d'arguments que prévu → les valeurs supplémentaires sont ignorées (sauf si on utilise ...rest).

```
1  function infos(name, age){
2    //afficher nom + age
3  }
4
5  infos("Léo"); //affiche "Léo"
6  infos("Léo", 25, true); //affiche
   "Léo, 25" true est ignoré
```

Paramètres par défaut

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

En Javascript, on peut assigner une valeur par défaut à un paramètre dans la déclaration d'une fonction.

Cela permet d'éviter que le paramètre soit undefined si aucun argument n'est fourni à l'appel de la fonction.

Sans valeur par défaut, un paramètre non fourni est automatiquement undefined. Cela peut entraîner :

- des messages incomplets (Bonjour, undefined !)
- des erreurs
- des comportements inattendus (par exemple, NaN dans des calculs)

```
1  function show(name = "John"){
2    //afficher prénom
3  }
4
5  show(); //prénom John
6  show("Paul"); //prénom Paul
```

Paramètres rest (...rest)

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Le paramètre rest (...nom) permet de récupérer un nombre variable d'arguments sous forme de tableau.

Il est très utile lorsque vous ne connaissez pas à l'avance le nombre d'arguments qu'un utilisateur pourrait passer à une fonction.

Dans l'exemple, chaque argument passé à la fonction sera ajouté dans un tableau nommé "**params**".

```
1  function myFunction(...params){
2    //afficher params1 param2 etc..
3  }
4
5  myFunction("Ali", "Marie", "Paul")
```

Passage par valeur et par référence

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

En Javascript, les types primitifs (comme les nombres, les chaînes de caractères, les booléens, undefined, null, symbol, et bigint) sont passés par valeur lorsqu'on les transmet en argument à une fonction.

Cela signifie que la fonction reçoit une copie indépendante de la variable d'origine. Toute modification à l'intérieur de la fonction ne change pas la variable extérieure.

```
1  function double(x){
2    x = x * 2;
3    //x donnera 20
4  }
5
6  let a = 10;
7  double(a);
8  //a donnera 10
```

Petite différence

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Les fonctions fléchés peuvent implicitement retourner une valeur.

Si le corps d'une fonction fléchée tient en une seule expression, le return est implicite.

Attention ! Ajouter des accolades désactive le return implicite.

```
1  const double = (x) => x * 2;
2  console.log(double(6)) //12
3
4  const double = (x) => { x * 2 };
5  console.log(double(6)) //undefined
6
7  //même code avec accolades
8  const double = (x) => { return x * 2 };
9  console.log(double(6)) //12
```